Imagine you are working at Intel in the 1980s

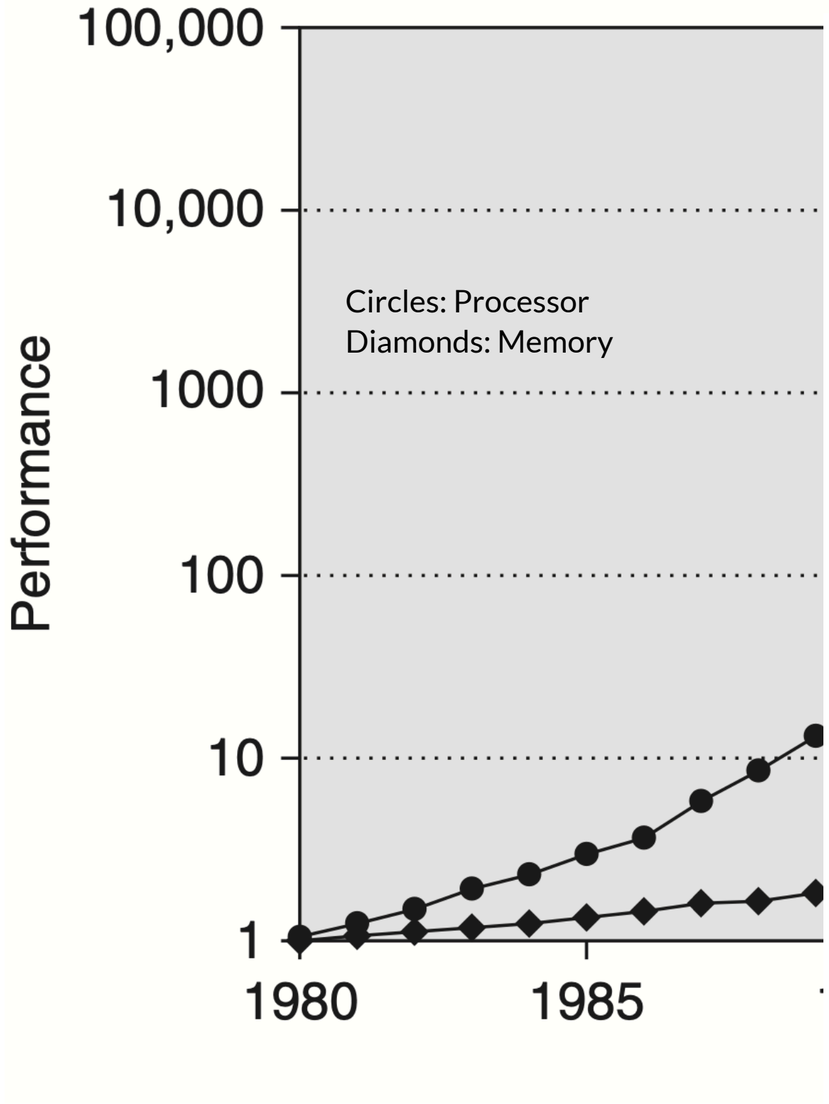


Image from “Computer Architecture: A Quantitative Approach”

Imagine you are working at Intel in the 1980s

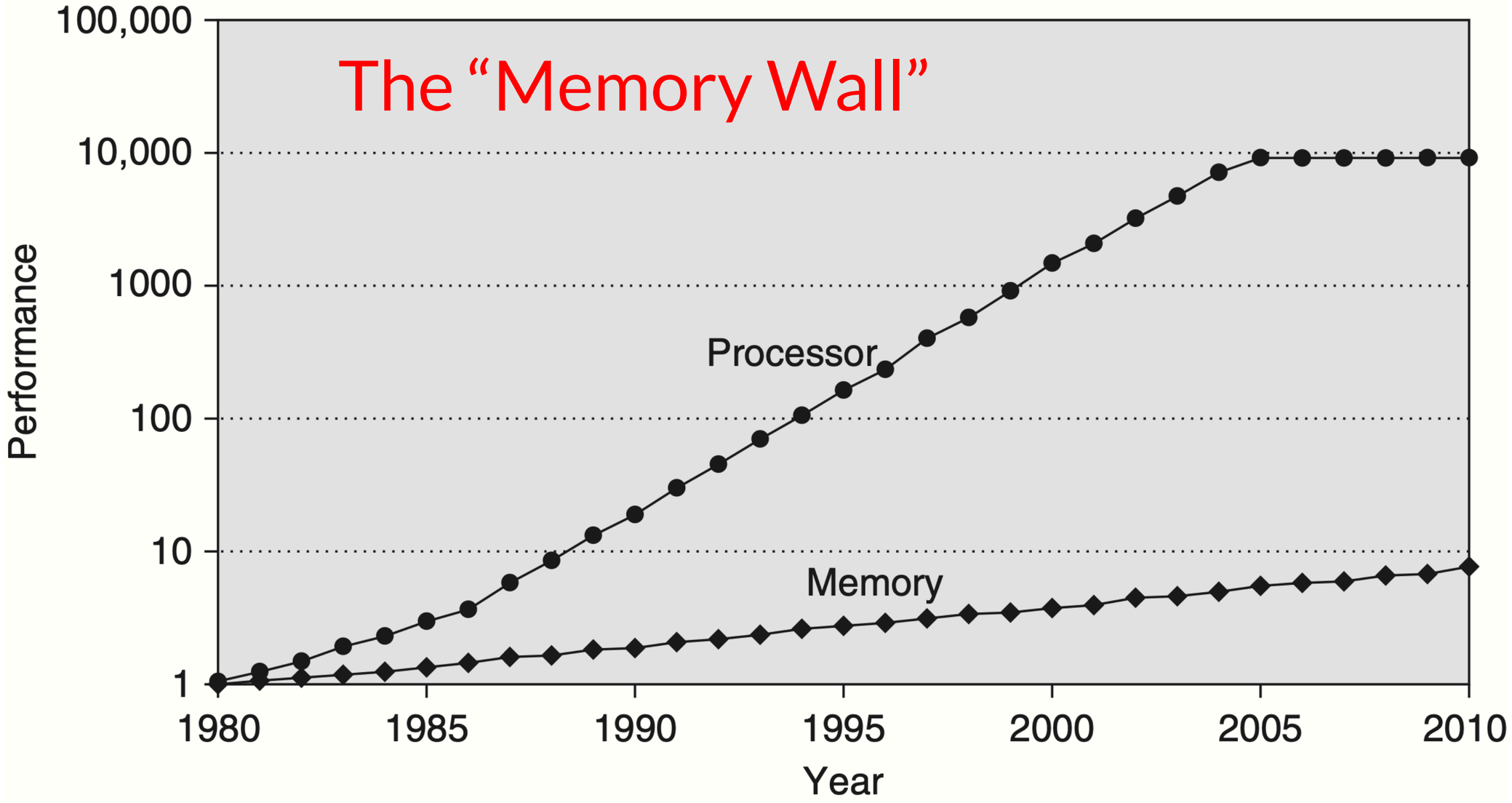


Image from “Computer Architecture: A Quantitative Approach”

Imagine you are working at Intel in the 1980s

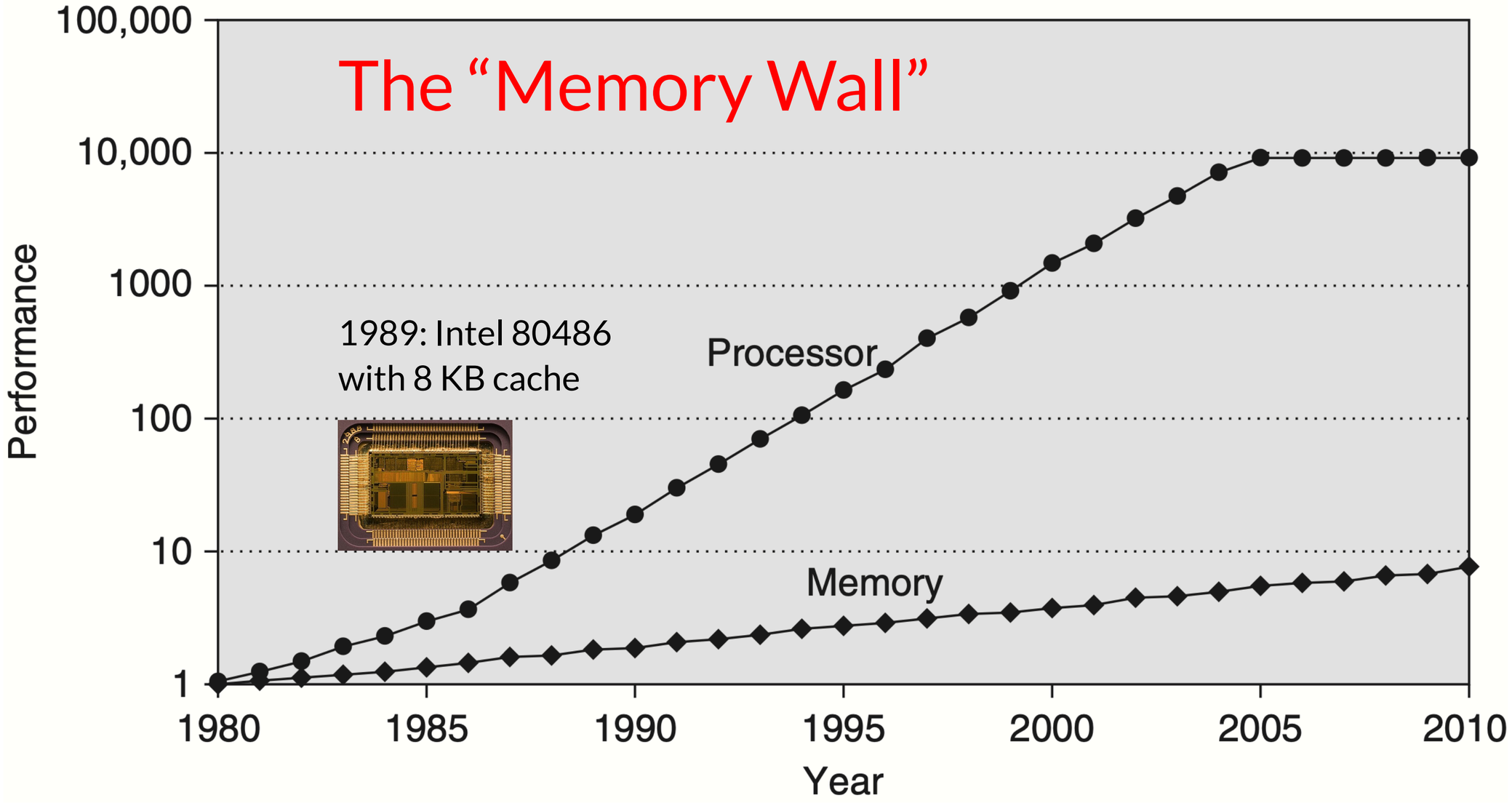
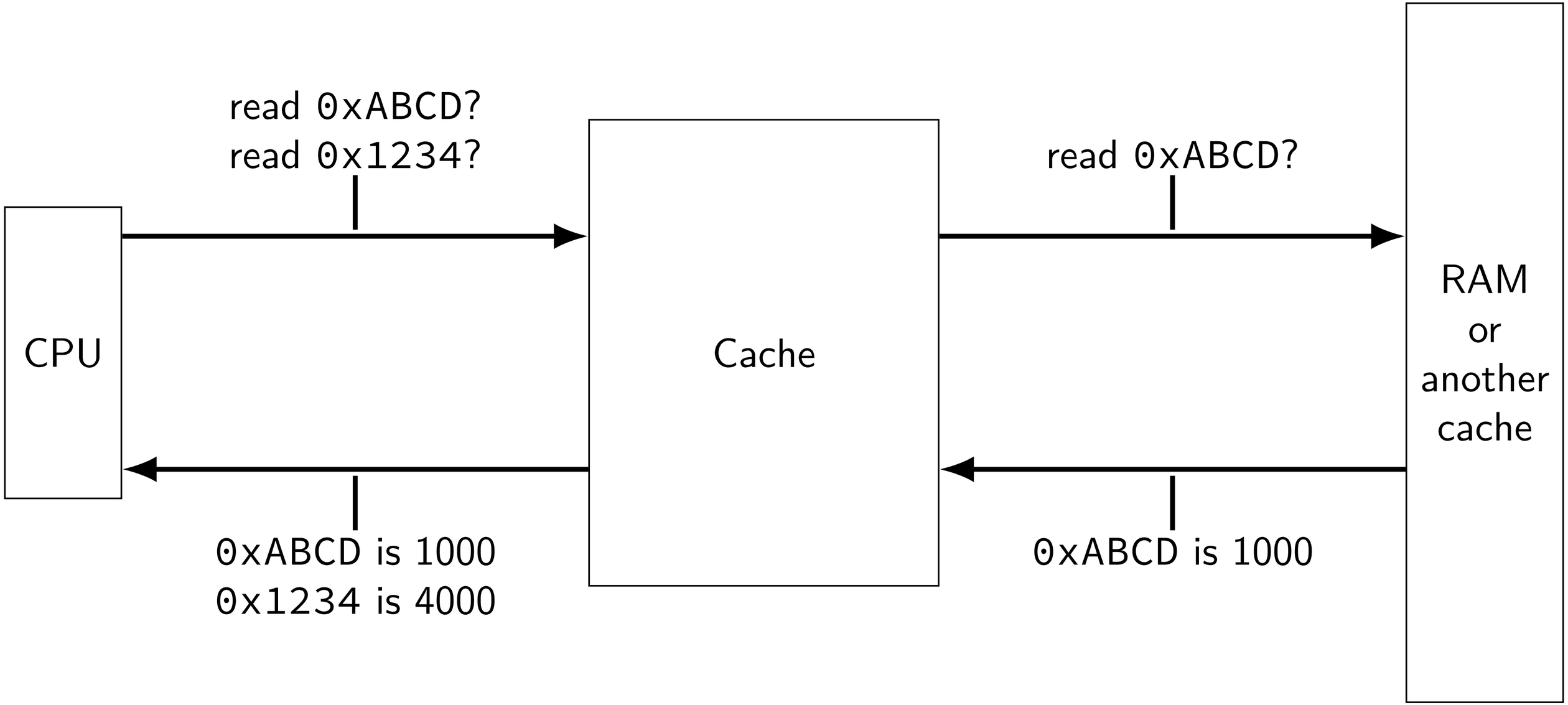
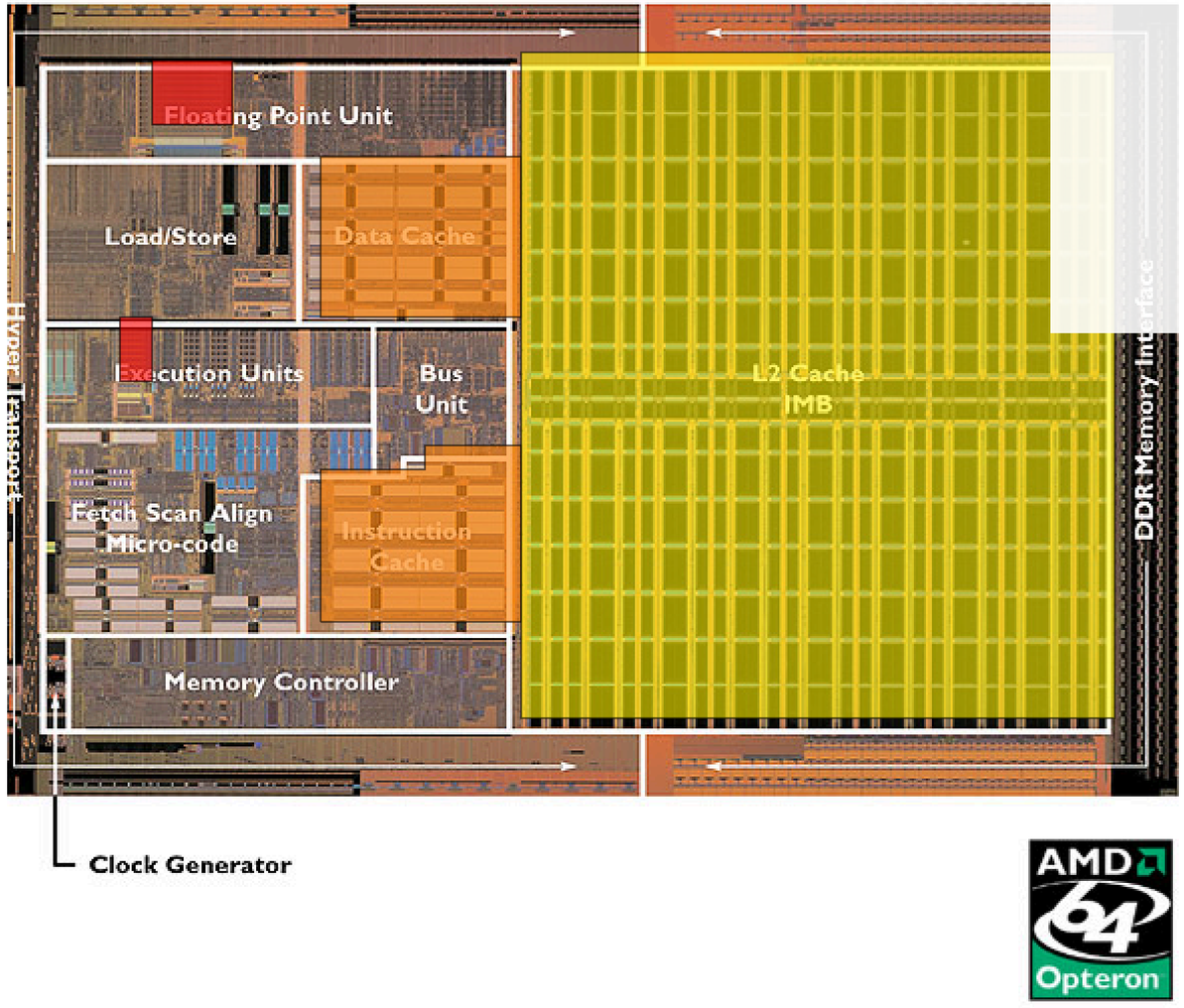


Image from "Computer Architecture: A Quantitative Approach"

the place of cache



memory hierarchy



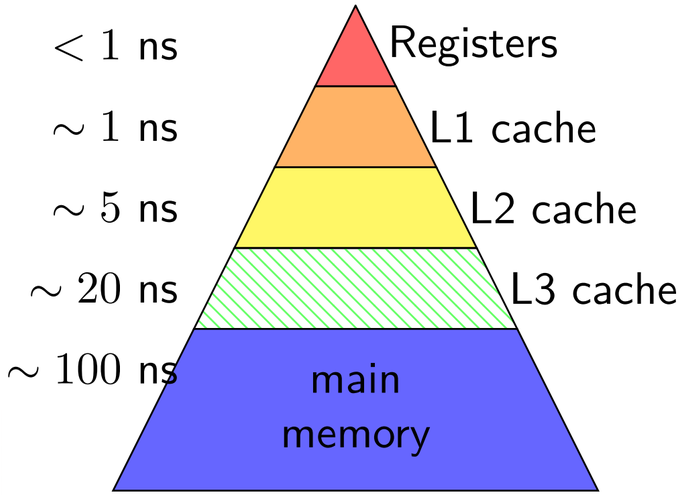


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

memory hierarchy goals

performance of the fastest (smallest) memory

capacity of the largest (slowest) memory

how to hide 100x latency difference?

99+% hit rate ("hit" means value found in cache)

memory hierarchy assumptions

temporal locality

"if a value is accessed now, it will be accessed again soon"

caches should keep *recently accessed values*

spatial locality

"if a value is accessed now, adjacent values will be accessed soon"

caches should *store adjacent values at the same time*

natural properties of programs — think about loops

locality examples

```
double computeMean(int length, double *values) {
    double total = 0.0;
    for (int i = 0; i < length; ++i) {
        total += values[i];
    }
    return total / length;
}
```

temporal locality: machine code of the loop

spatial locality: machine code of most consecutive instructions

temporal locality: `total`, `i`, `length` accessed repeatedly

spatial locality: `values[i+1]` accessed after `values[i]`

locality exercise (1)

```
/* version 1 */
for (int i = 0; i < N; ++i)
    for (int j = 0; j < N; ++j)
        A[i] += B[j] * C[i * N + j]

/* version 2 */
for (int j = 0; j < N; ++j)
    for (int i = 0; i < N; ++i)
        A[i] += B[j] * C[i * N + j];
```

exercise: which has better temporal locality in A? in B? in C?

how about spatial locality?

locality exercise (2)

```
/* version 1 */
for (int i = 0; i < N; ++i)
    for (int j = 0; j < N; ++j)
        A[i] += B[j] * C[i * N + j]

/* version 3 */
for (int ii = 0; ii < N; ii += 32)
    for (int jj = 0; jj < N; jj += 32)
        for (int i = ii; i < ii + 32; ++i)
            for (int j = jj; j < jj + 32; ++j)
                A[i] += B[j] * C[i * N + j];
```

exercise: which has better temporal locality in A? in B? in C?

how about spatial locality?

locality exercise (3)

```
struct Student {
    char name[128]; long id; float grade;
};
struct Student students[1000];

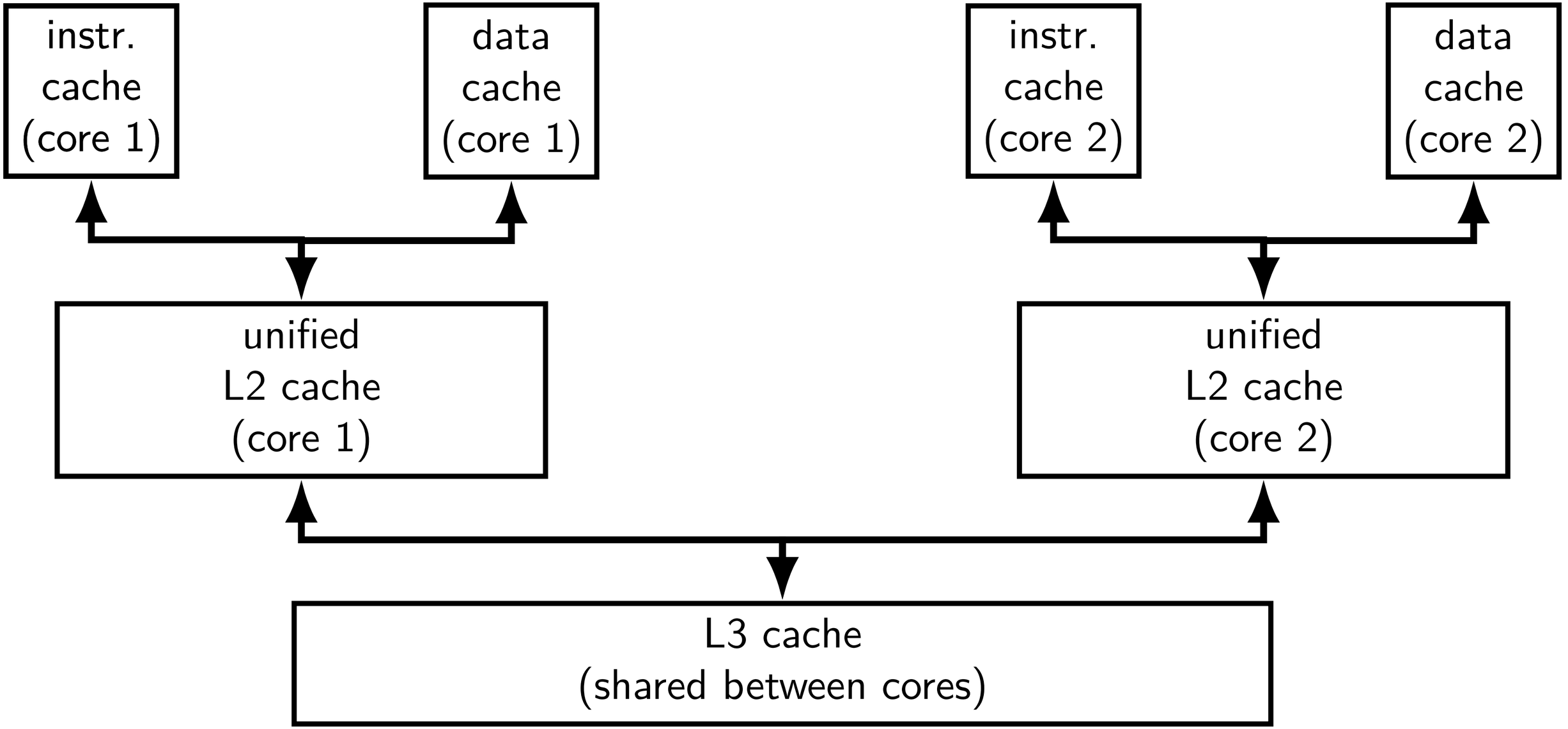
float averageGrade() {
    float sum = 0.0
    for (int i = 0; i < 1000; i += 1)
        sum += students[i].grade;
    return sum / 1000.0;
}
```

```
char studentNames[1000][128];
long studentIds[1000];
float studentGrades[1000];

float average() {
    float sum = 0.0
    for (int i = 0; i < 1000; i += 1)
        sum += studentGrades[i];
    return sum / 1000.0;
}
```

better spatial/temporal locality?

split caches; multiple cores (one design)



hierarchy and instruction/data caches

typically separate data and instruction caches for L1

(almost) never going to read instructions as data or vice-versa

avoids instructions evicting data and vice-versa

can optimize instruction cache for different access pattern

easier to build fast caches: handle fewer accesses at a time

cache analogy: you finding words in books

library: main memory

- contains all the words in all books

copy of books you took home: L2 cache

- faster to look in books already at home

copy of individual book pages in your backpack: L1 cache

- fastest to look at book pages you have with you

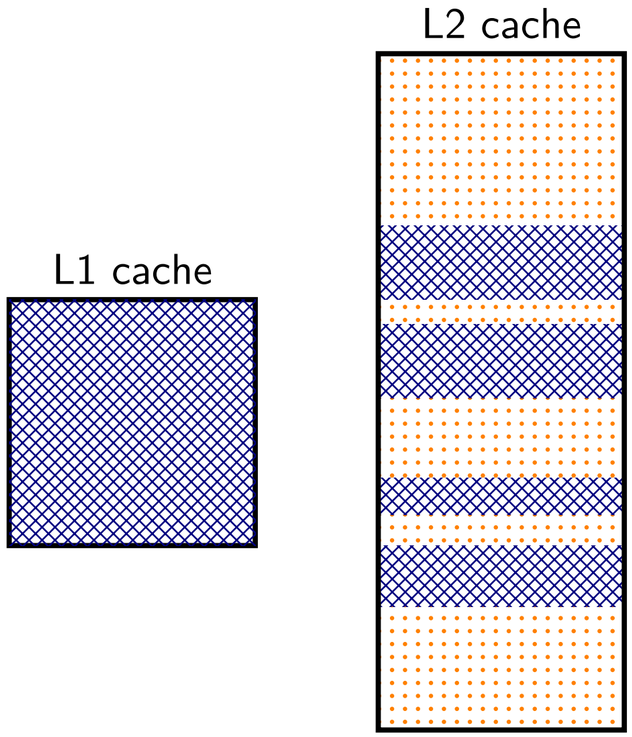
backup slides

inclusive versus exclusive

inclusive versus exclusive

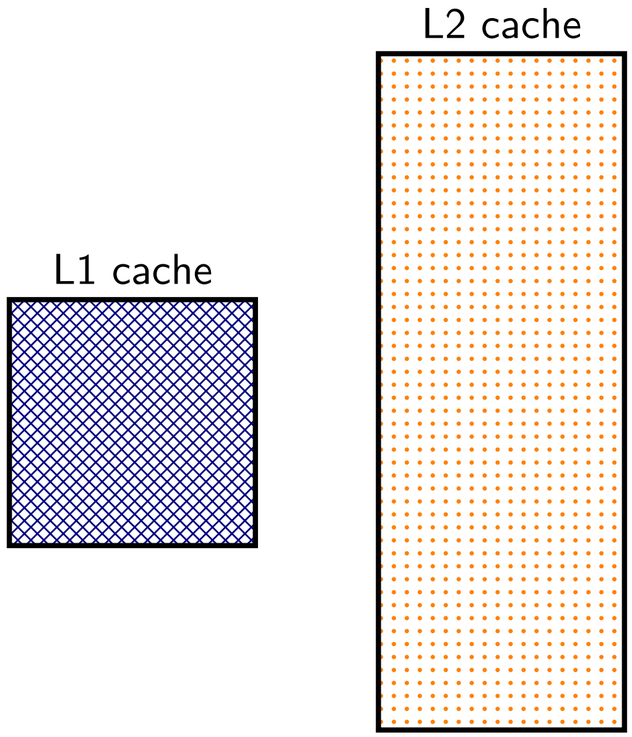
L2 inclusive of L1

everything in L1 cache duplicated in L2
adding to L1 also adds to L2



L2 exclusive of L1

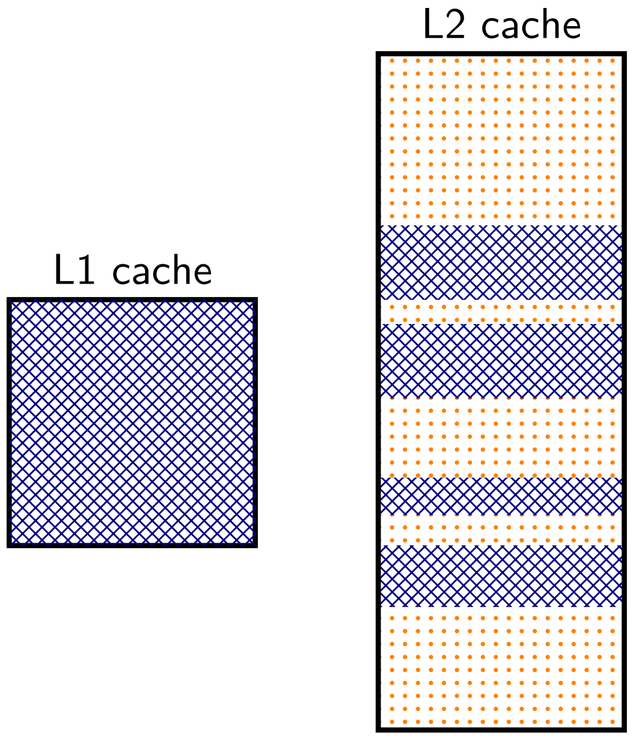
L2 contains different data than L1
adding to L1 must remove from L2
probably evicting from L1 adds to L2



inclusive versus exclusive

L2 inclusive of L1

everything in L1 cache duplicated in L2
adding to L1 also adds to L2



inclusive policy:
no extra work on eviction
but duplicated data

easier to explain when
Lk shared by multiple L$(k-1)$ caches?

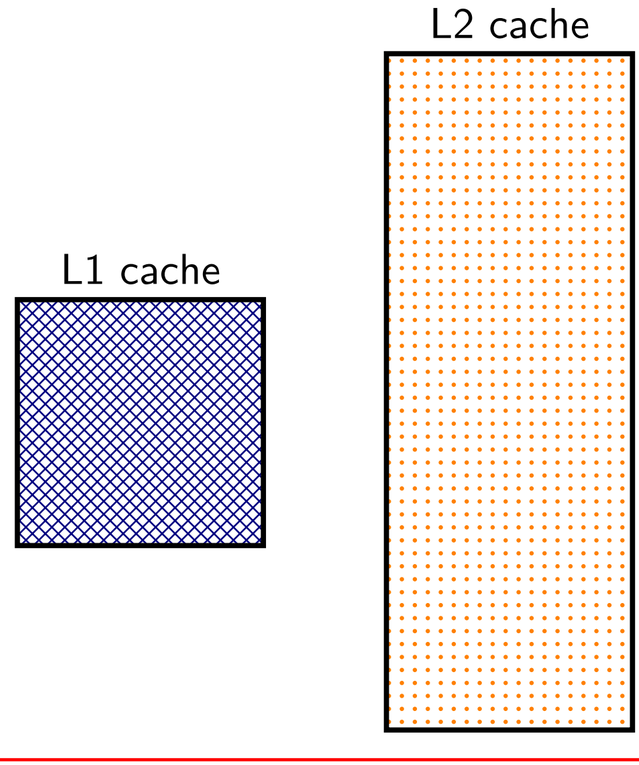
inclusive versus exclusive

exclusive policy:
avoid duplicated data
sometimes called *victim cache*
(contains cache eviction victims)

makes less sense with multicore

L2 exclusive of L1

L2 contains different data than L1
adding to L1 must remove from L2
probably evicting from L1 adds to L2

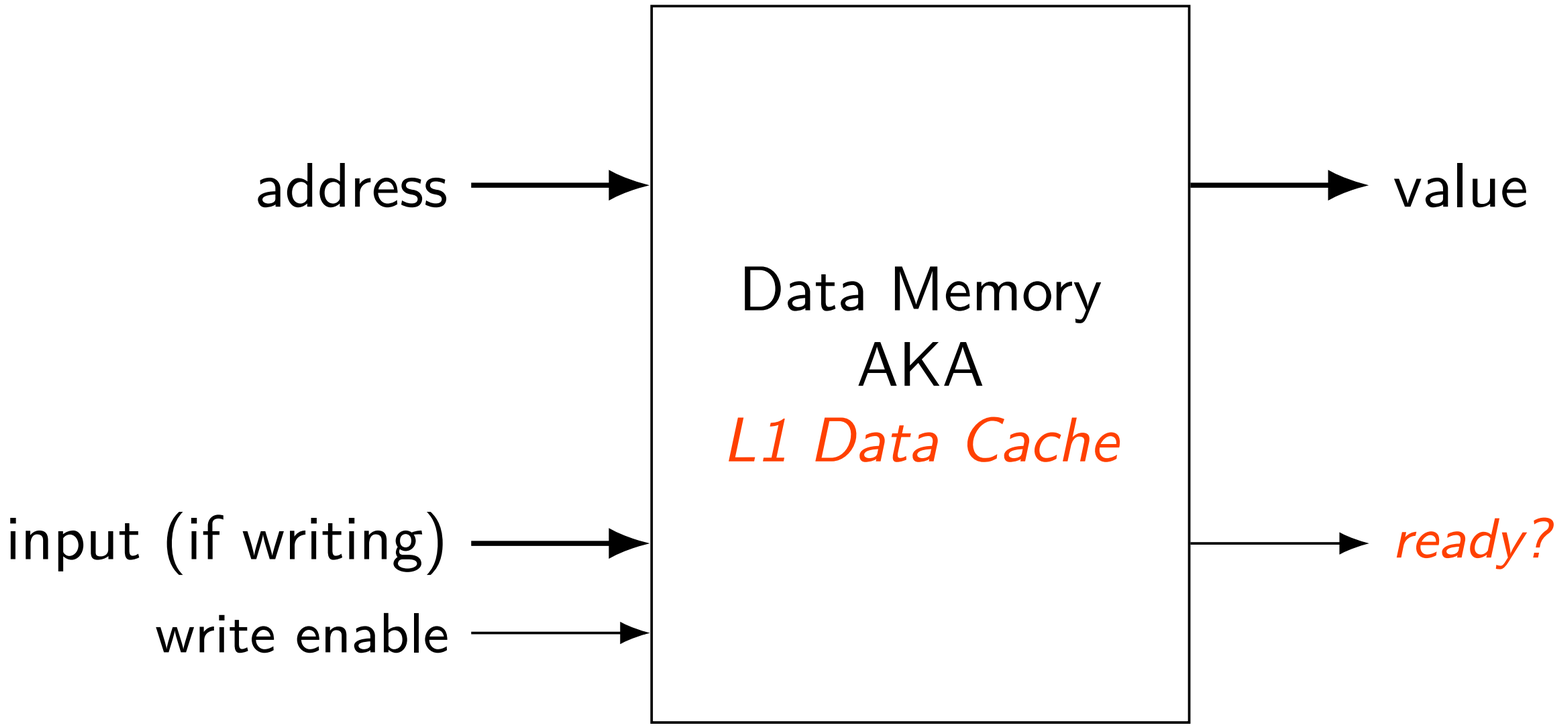


[fragile,label=loopOrderLocalExloop orders and locality

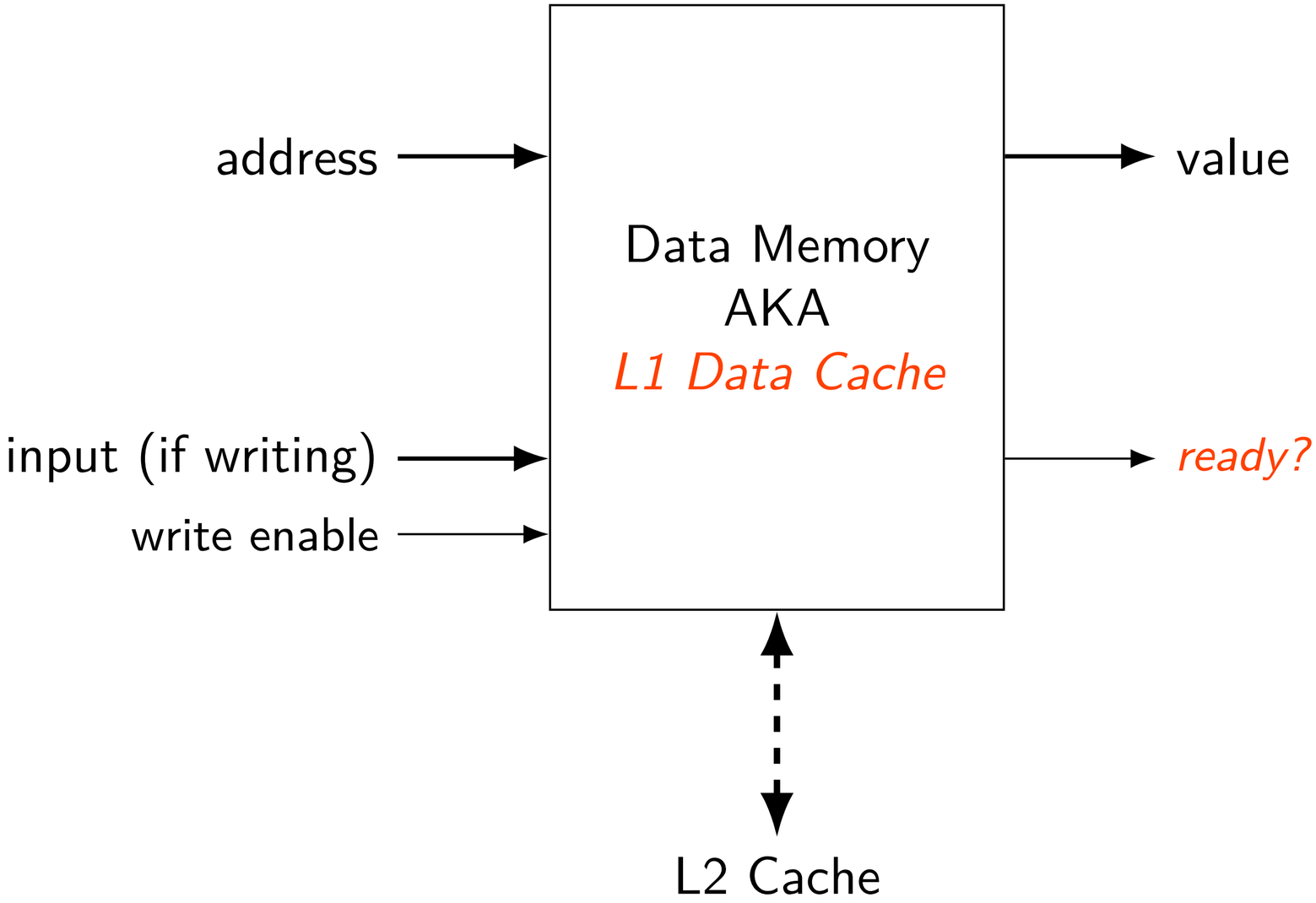
```
for (int i = 0; i < N; ++i)
    for (int j = 0; j < N; ++i)
        A[i * N] = B[j];
```

cache: real memory

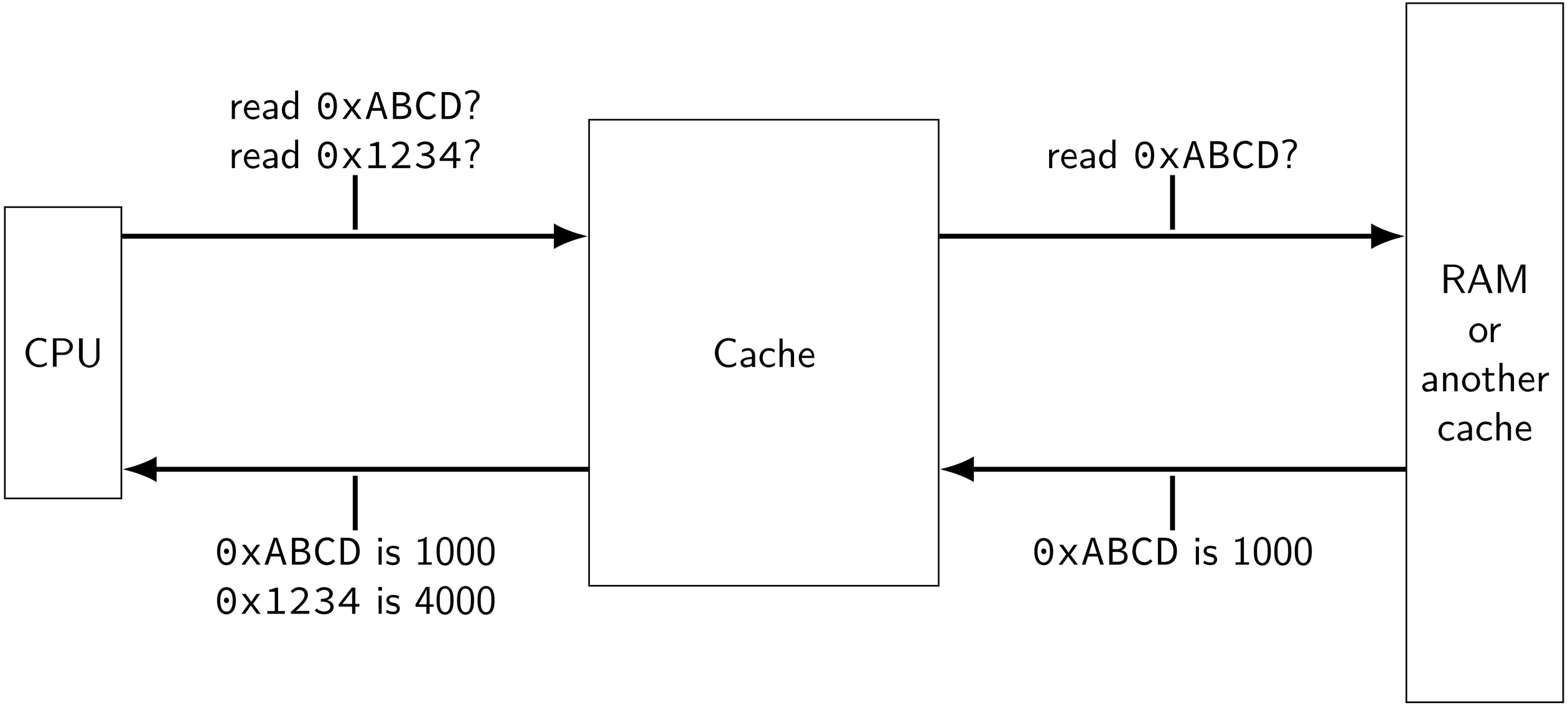
cache: real memory



cache: real memory



the place of cache



2004 CPU

Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU

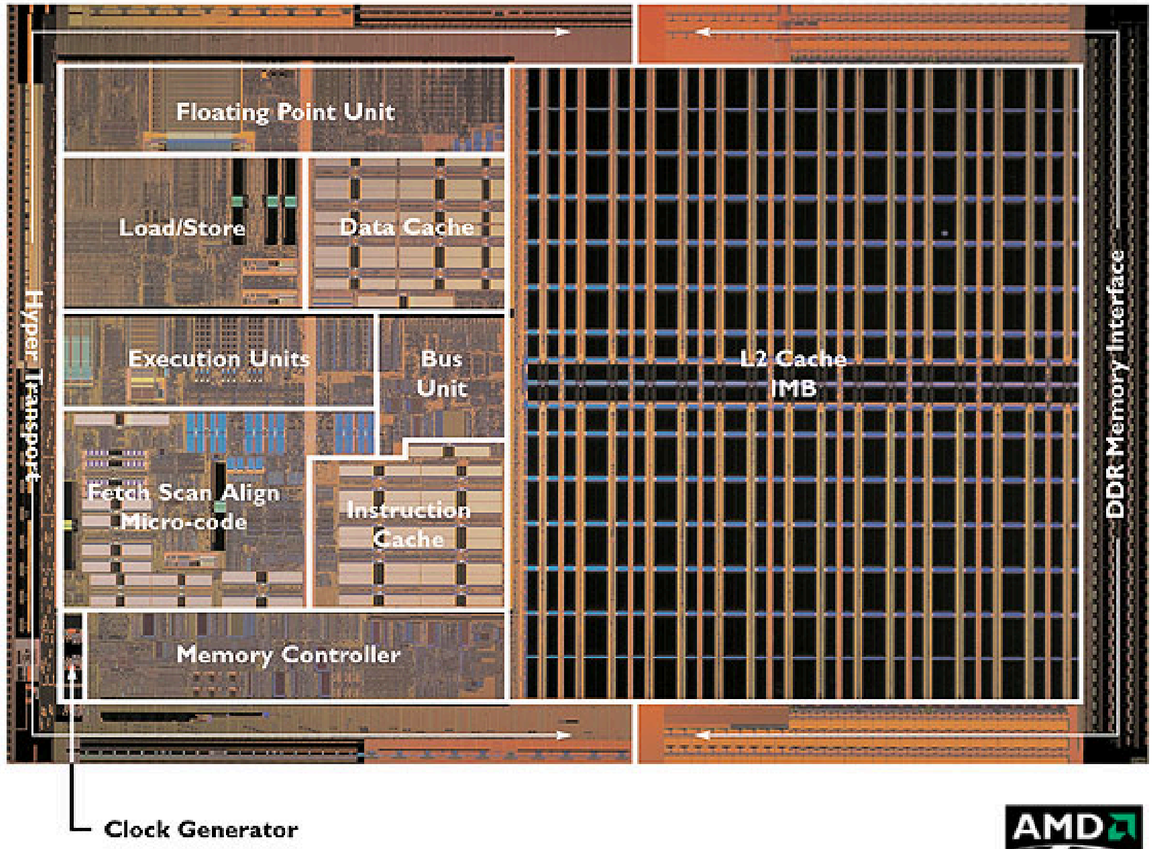
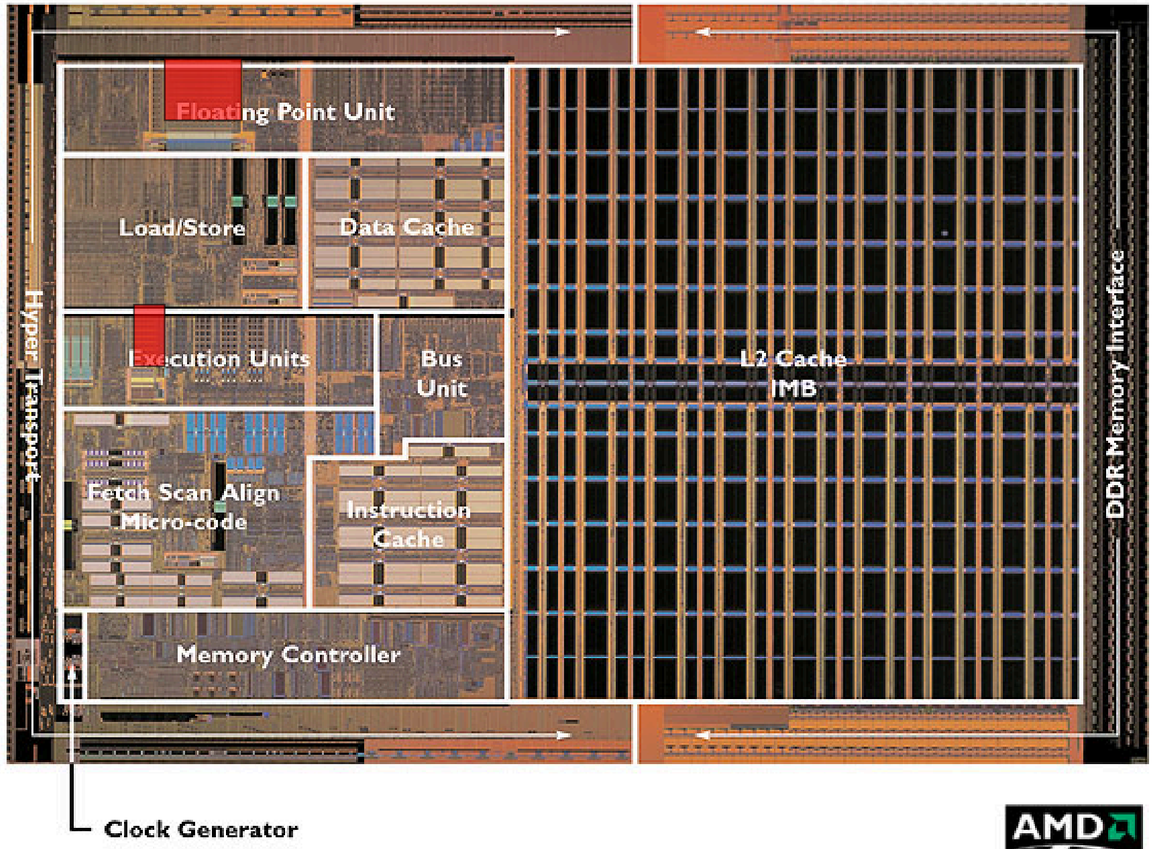


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

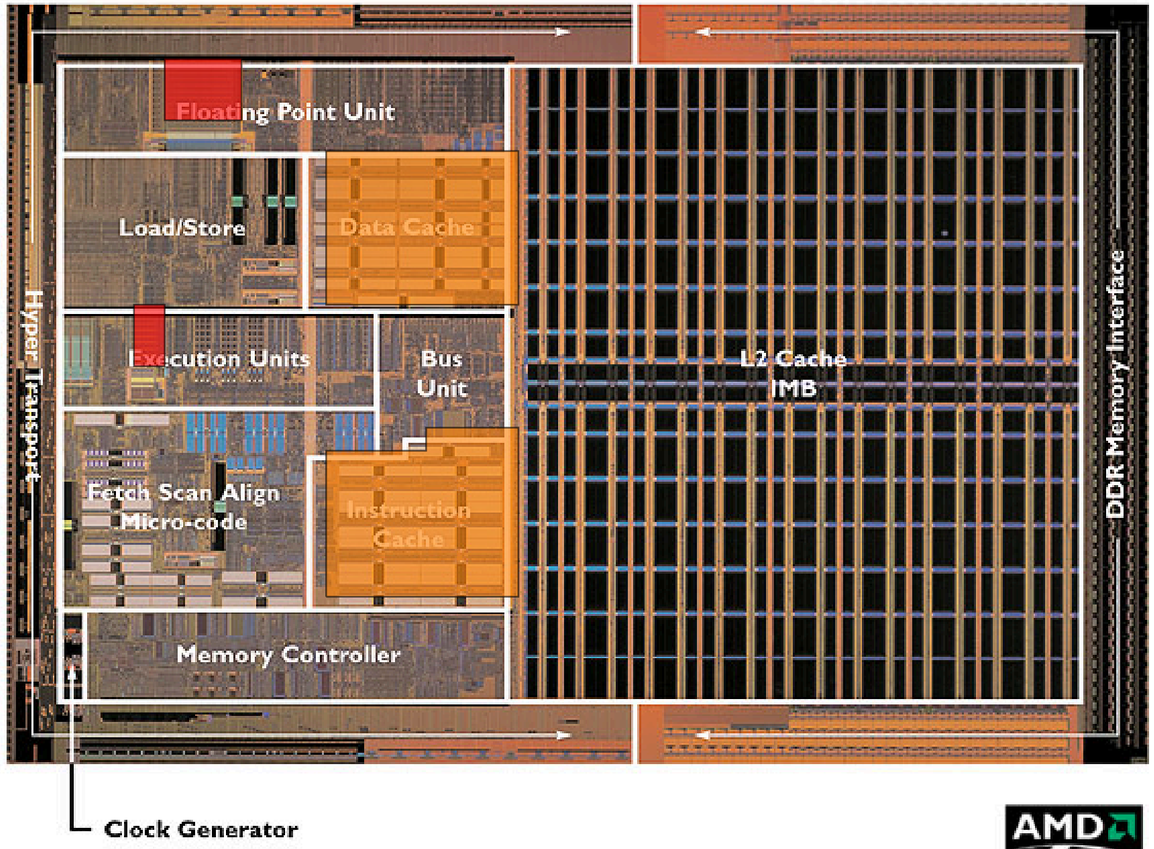
2004 CPU

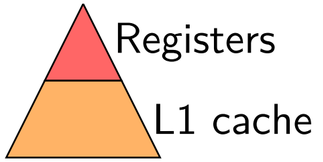


Registers

Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU

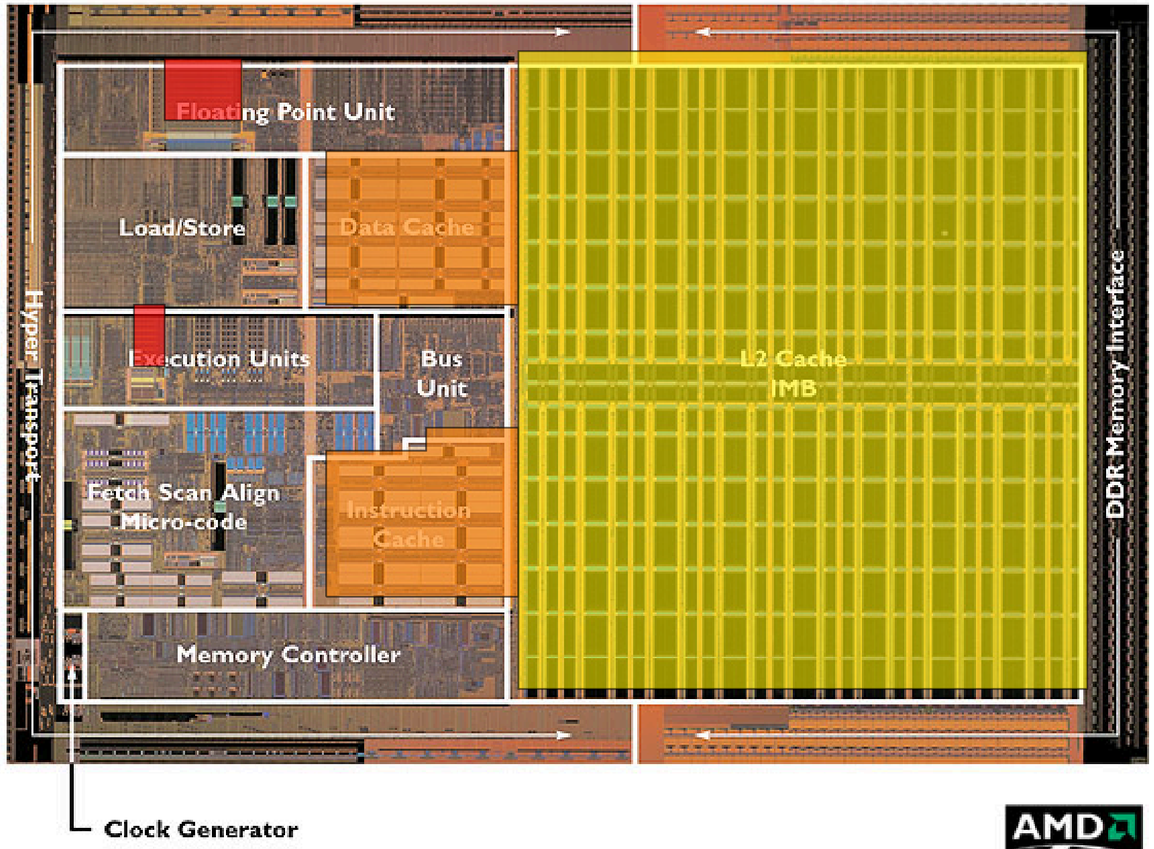




AMD 64 Opteron

Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



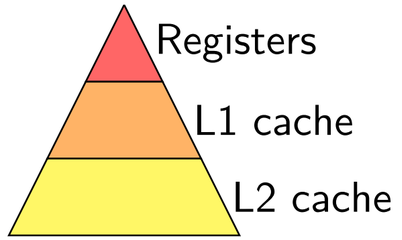
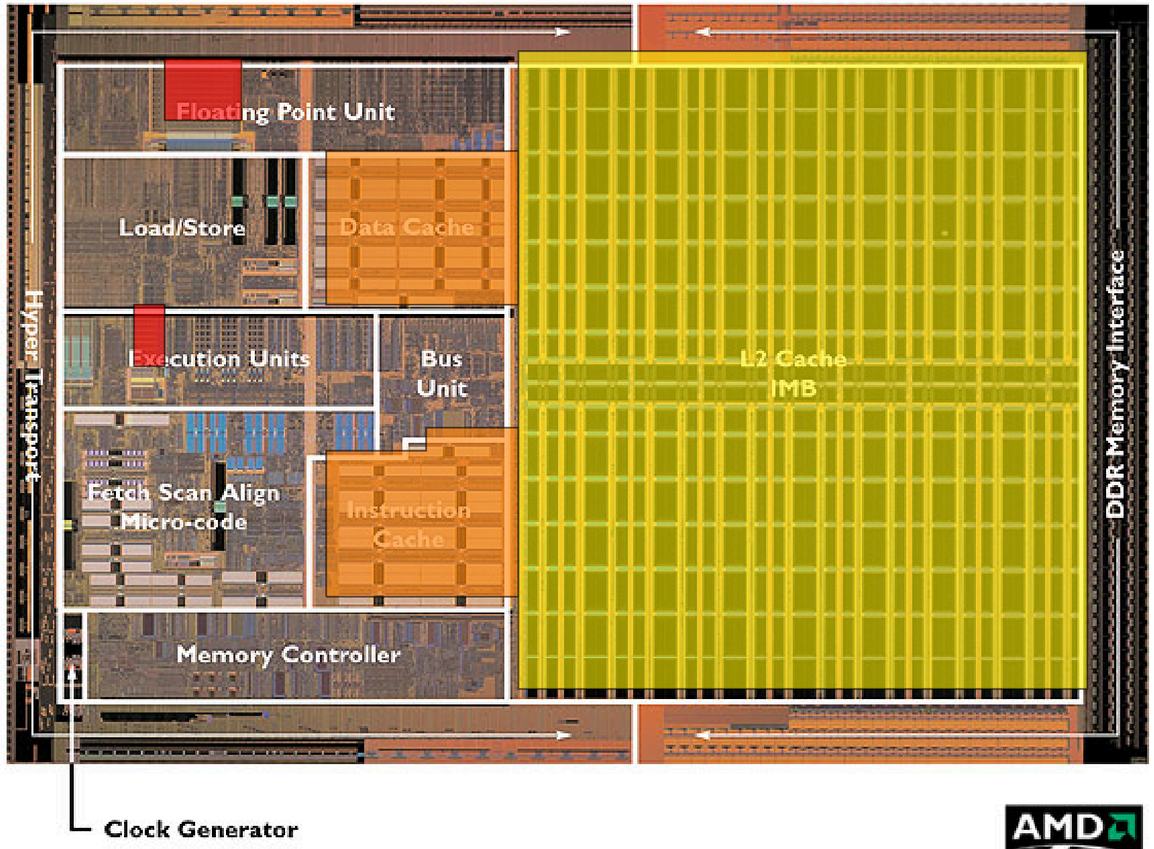


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



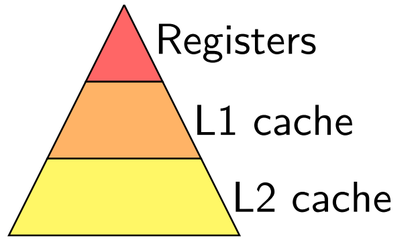
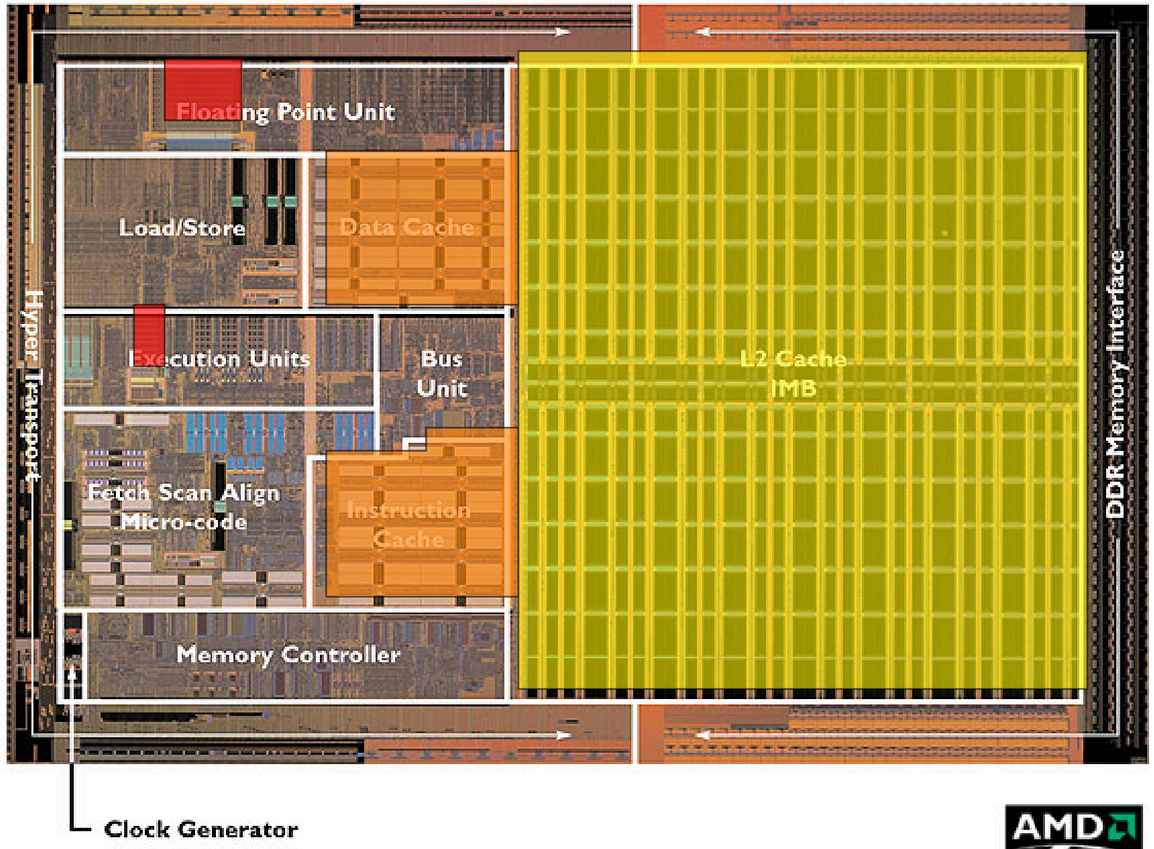


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



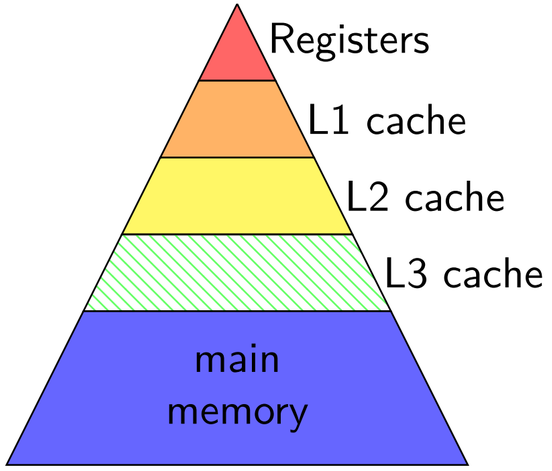
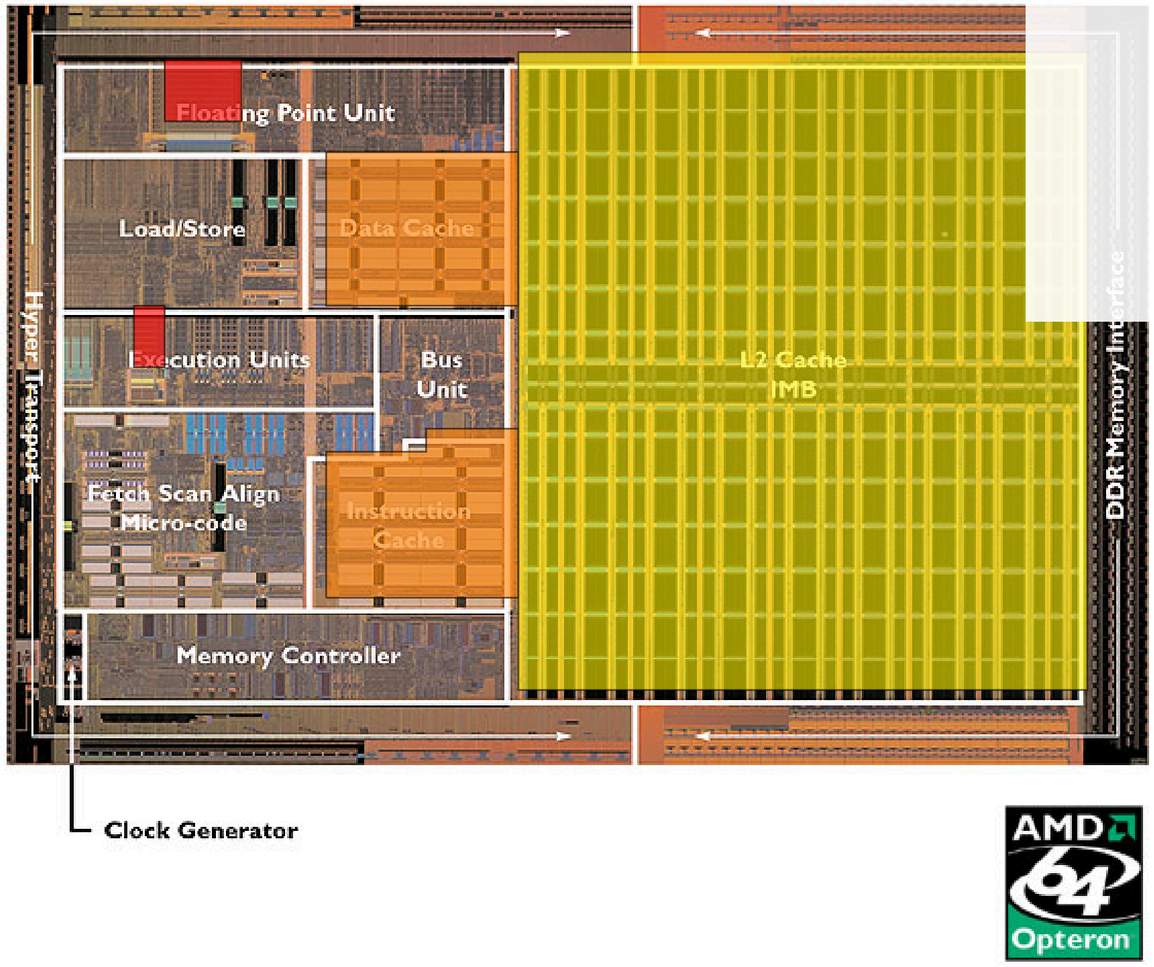


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



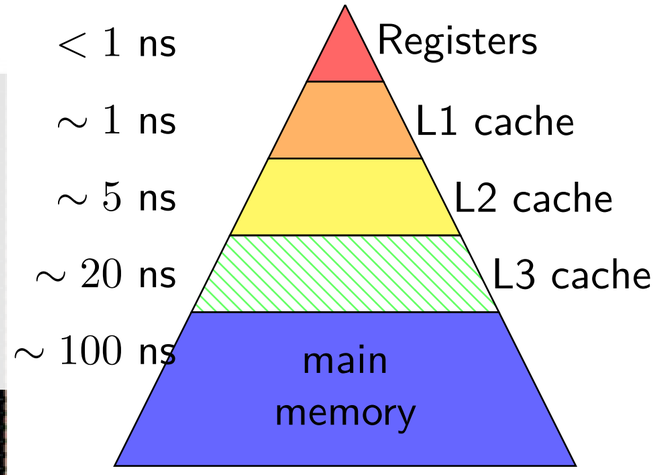
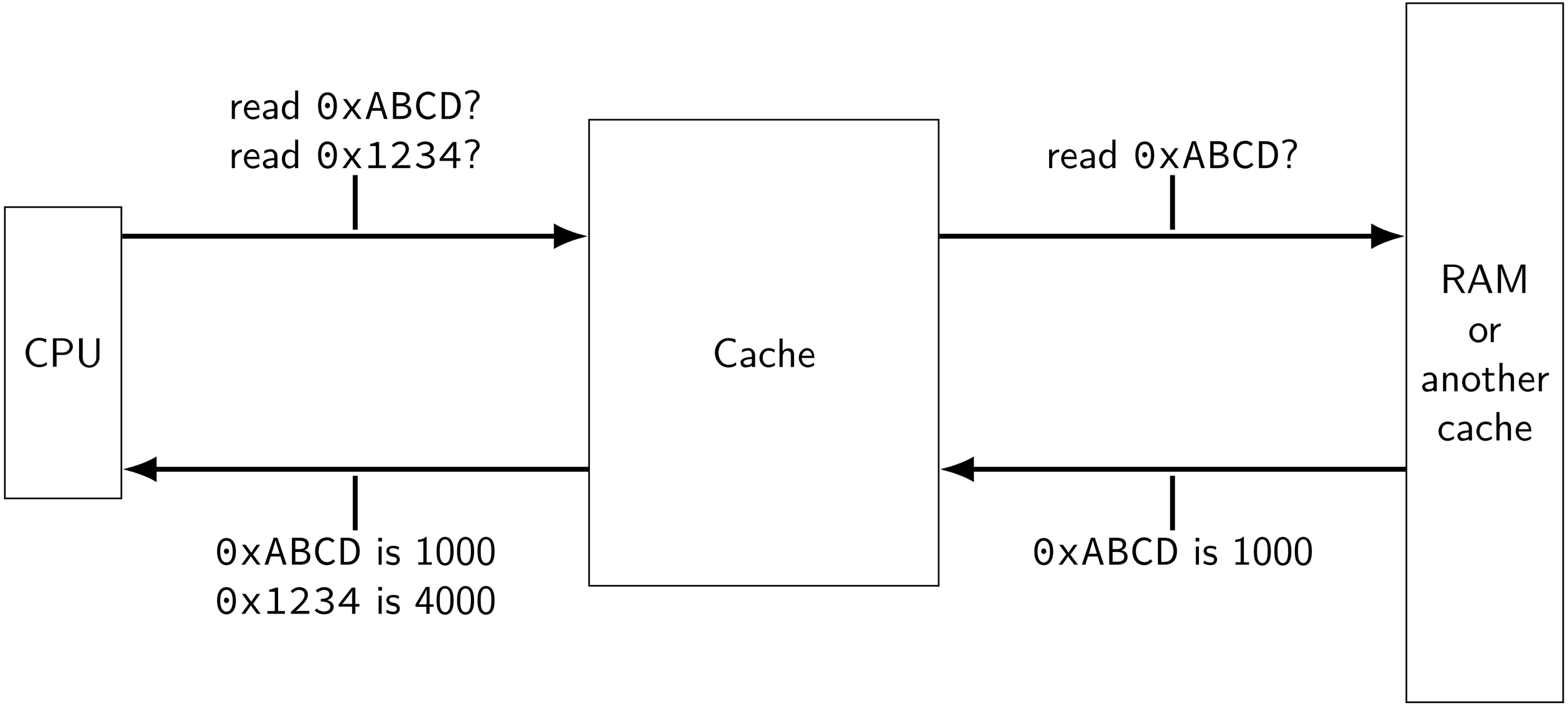


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

the place of cache (1)



memory hierarchy goals

performance of the fastest (smallest) memory

- hide 100x latency difference? *99+% hit (= value found in cache) rate*

capacity of the largest (slowest) memory

2004 CPU

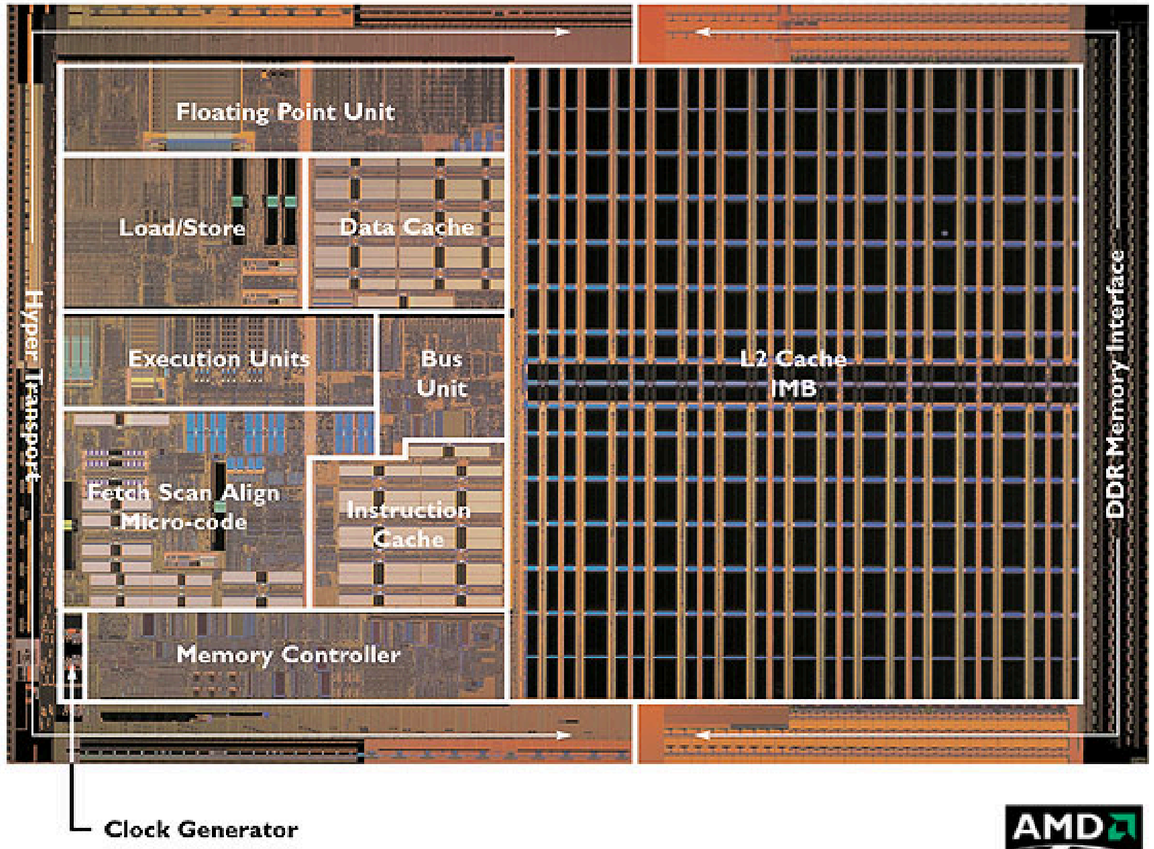
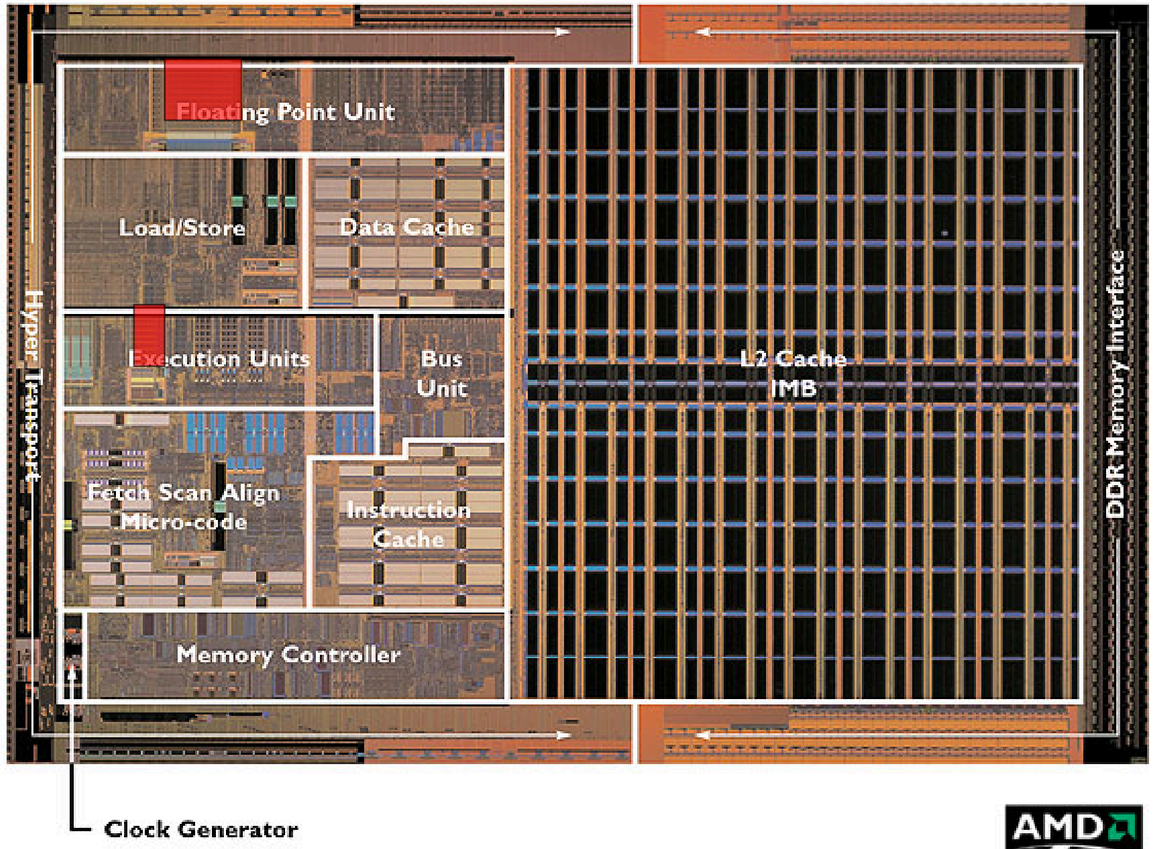


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

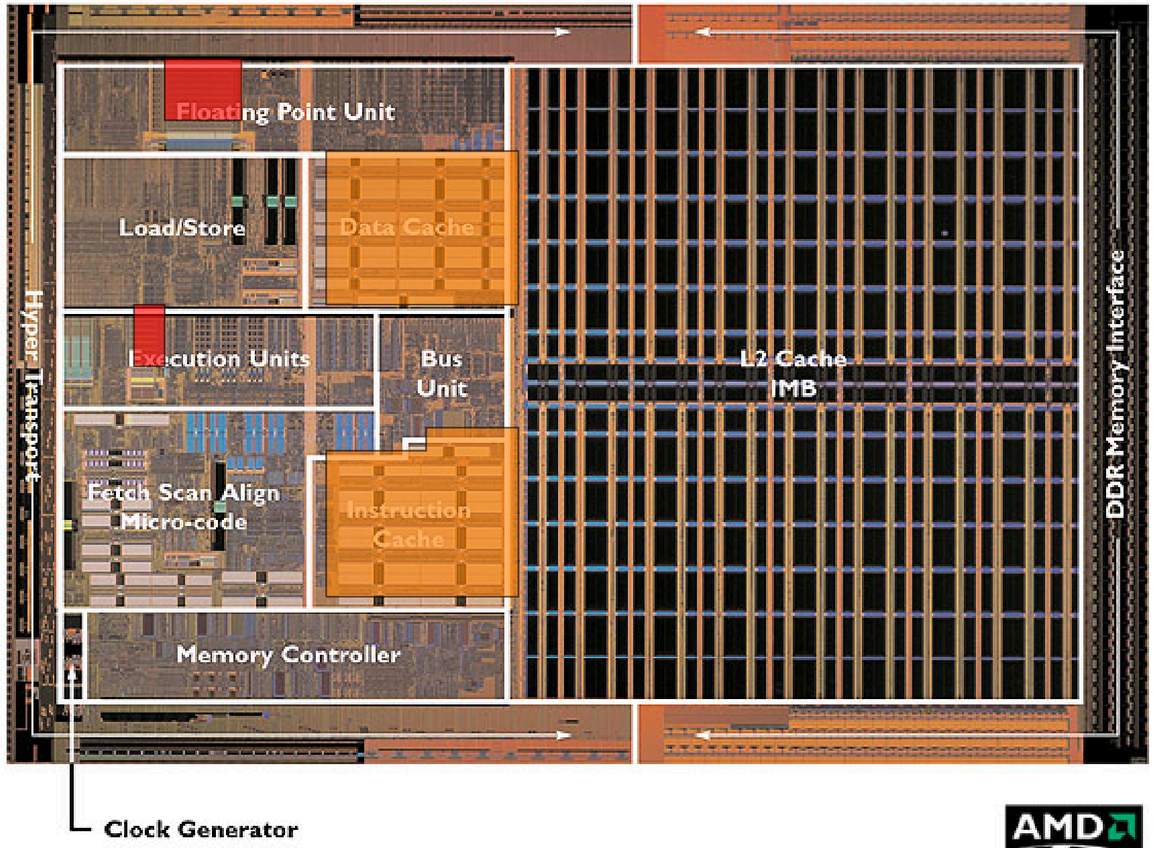
2004 CPU



Registers

Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



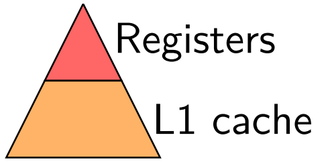
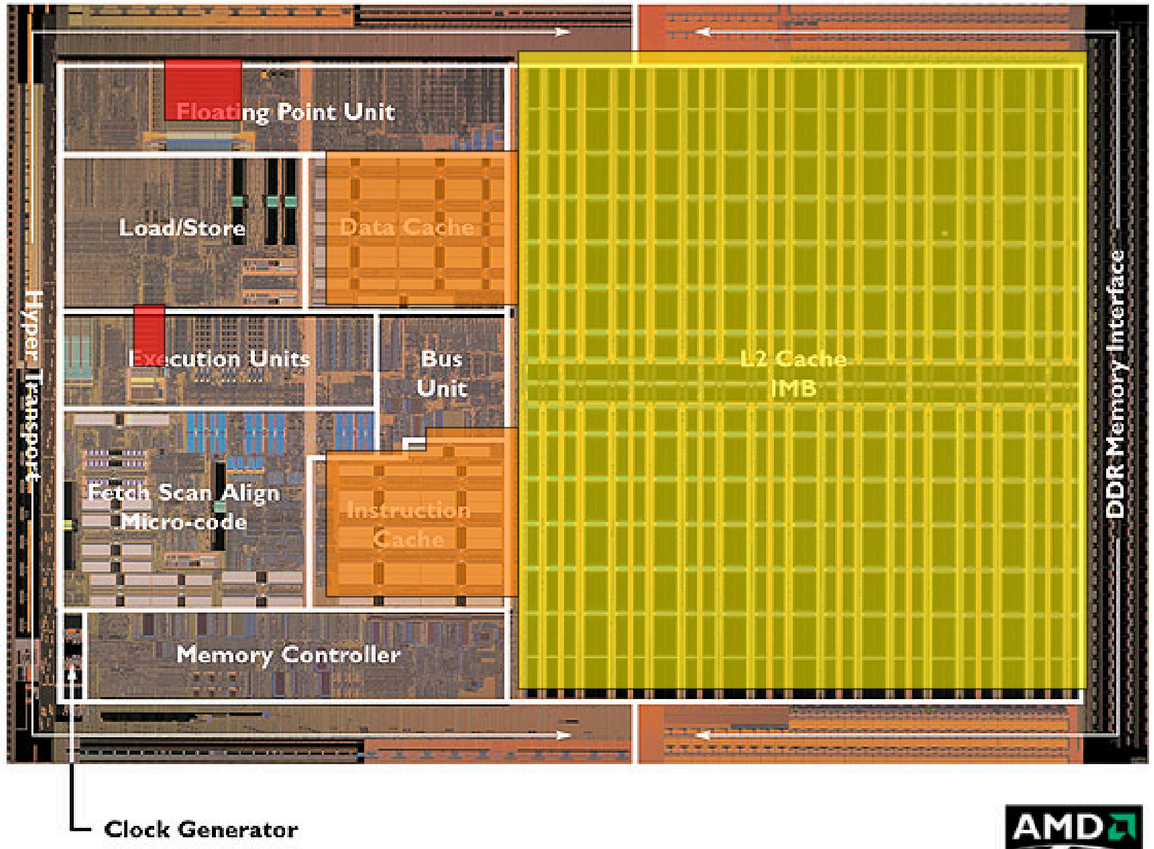


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



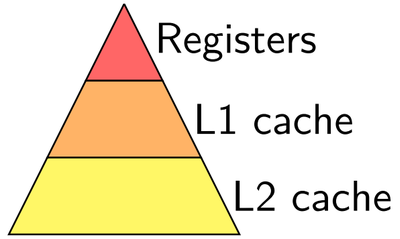
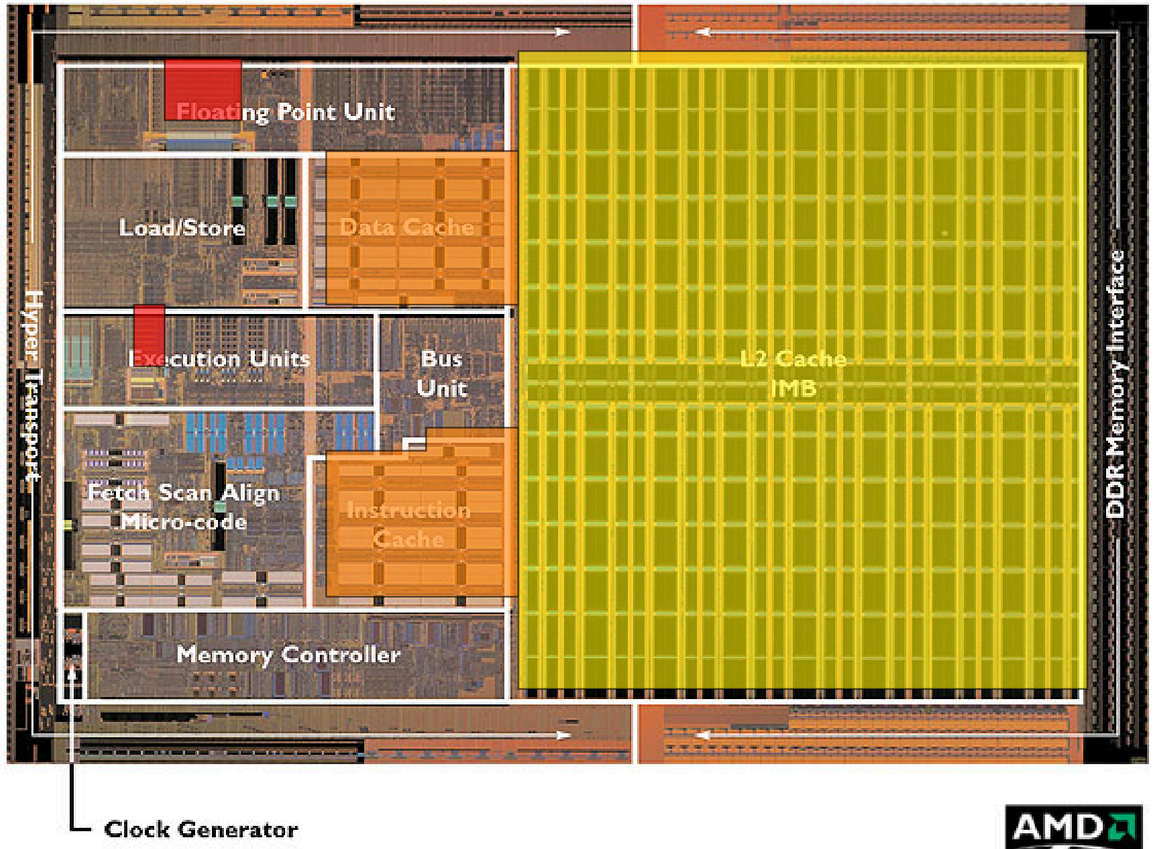


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



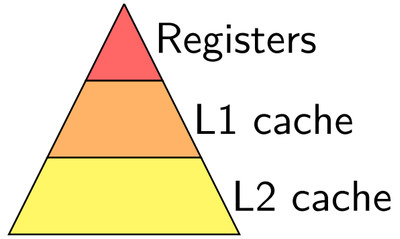
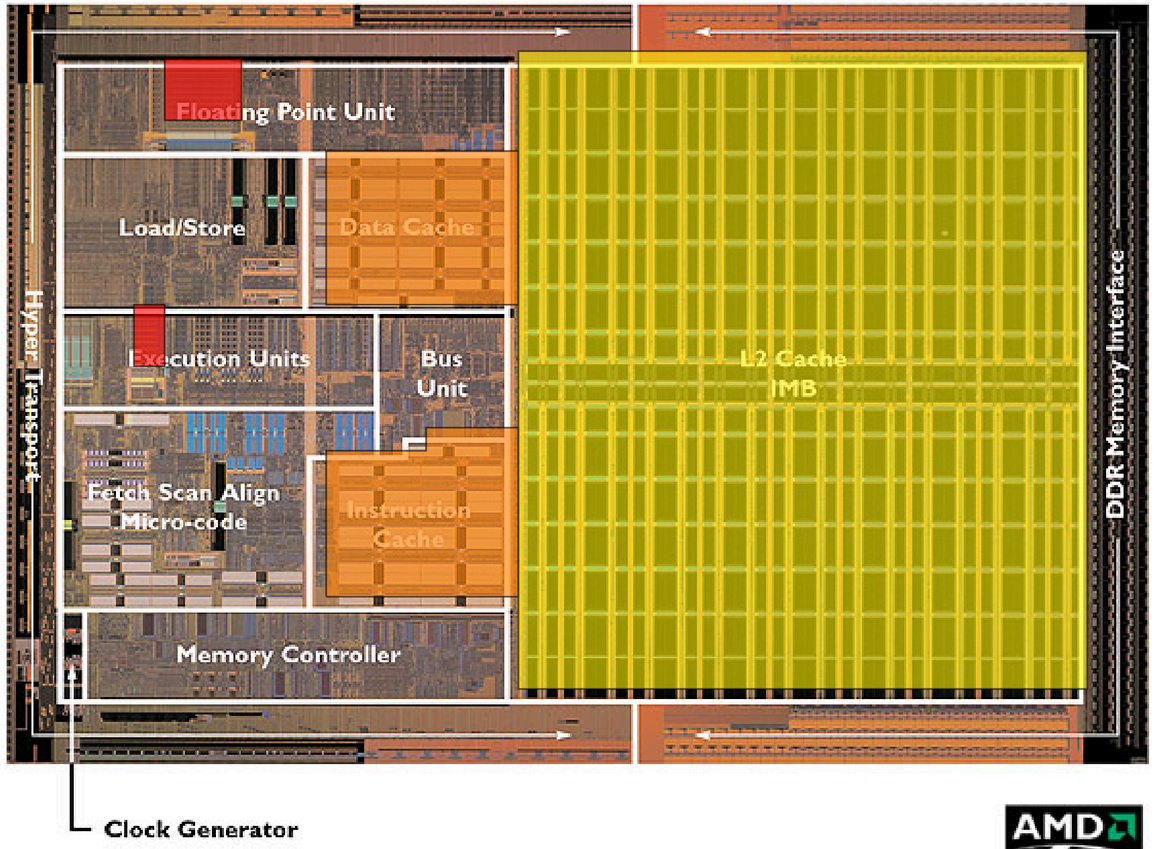


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



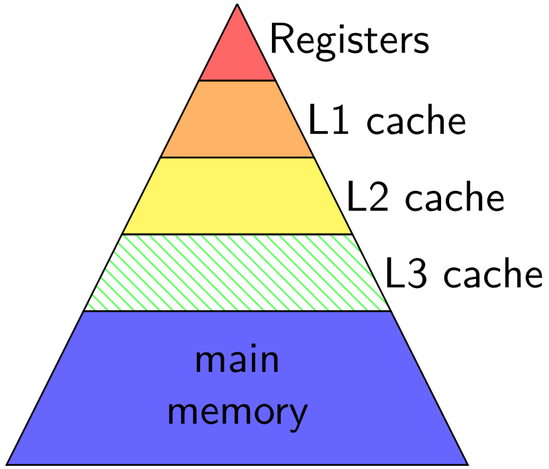
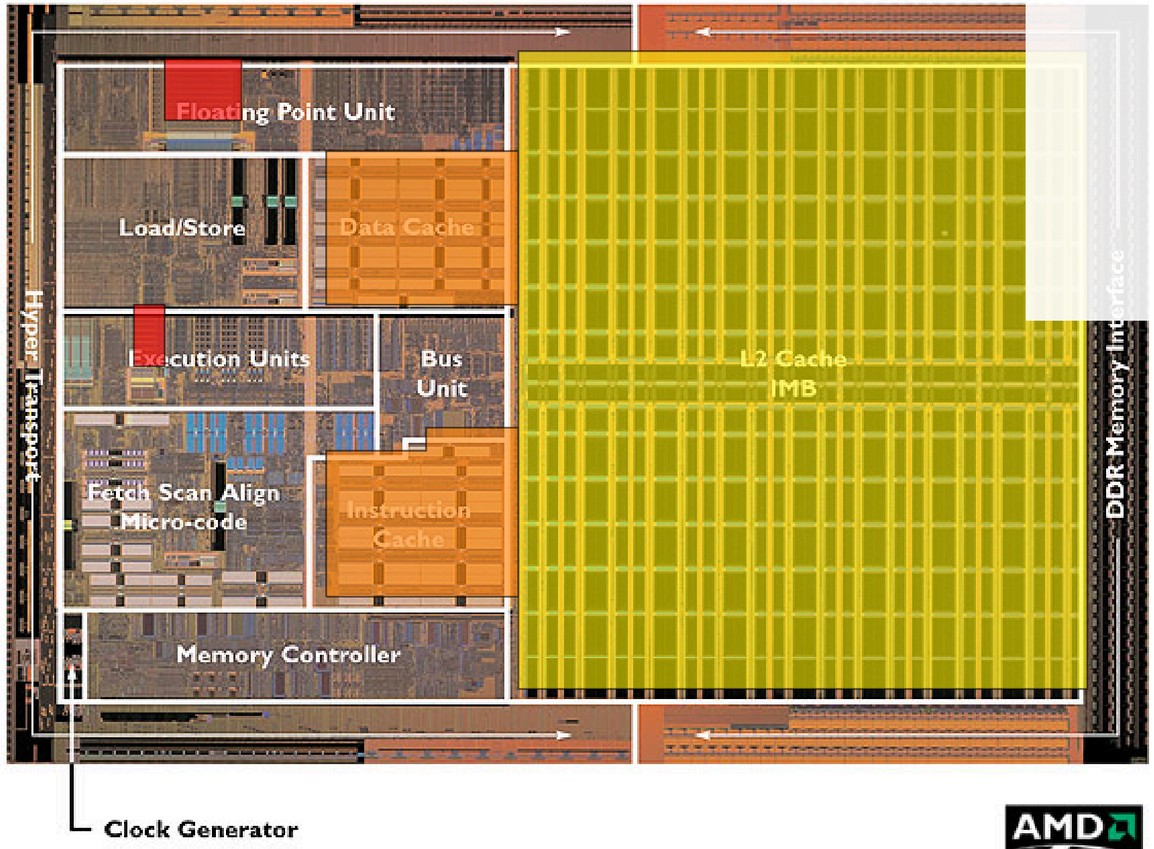


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)

2004 CPU



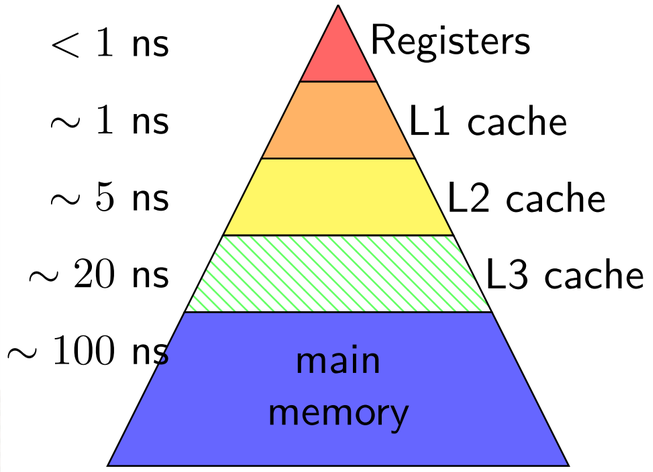


Image: approx 2004 AMD press image of Opteron die;
approx register location via chip-architect.org (Hans de Vries)